

MODULE -7

COLLECTIONS,FUNCTIONS&MOD ULES

Accessing List

Theory:

Q1. Understanding how to create and access elements in a list.

Understanding how to create and access elements within a list is fundamental in many programming languages, especially Python. Here's a breakdown of the key concepts:

1. Creating a List:

- . Basic Creation:**

- Lists are created using square brackets [], with elements separated by commas.**

- Lists can contain elements of various data types (integers, strings, floats, etc.).
- Example:
 - `my_list = [1, "hello", 3.14, True]`
 - `number_list = [1, 2, 3, 4, 5]`
 - `string_list = ["apple", "banana", "cherry"]`
- **Empty Lists:**
 - You can create an empty list like this: `empty_list = []`

2. Accessing Elements:

- **Indexing:**
 - Each element in a list has an index, starting from 0 for the first element.
 - You access elements using their index within square brackets.
 - Example:
 - `my_list[0]` would return 1 (the first element).

- `string_list[2]` would return "cherry" (the third element).
- **Negative Indexing:**
 - You can also use negative indexes to access elements from the end of the list.
 - -1 refers to the last element, -2 to the second last, and so on.
 - **Example:**
 - `my_list[-1]` would return True (the last element).
 - `string_list[-2]` would return "banana".
- **Slicing:**
 - Slicing allows you to extract a portion of a list.
 - You use the colon `:` operator to specify a range of indexes.
 - **Example:**

- `number_list[1:4]` would return `[2, 3, 4]` (elements from index 1 up to, but not including, index 4).
- `number_list[:3]` would return `[1, 2, 3]` (elements from the beginning up to, but not including, index 3).
- `number_list[3:]` would return `[4, 5]` (elements from index 3 to the end).
- `number_list[:]` would return a copy of the whole list.

Q2. Indexing in lists (positive and negative indexing).

When working with lists (or other sequence types like strings and tuples) in Python, indexing is how you access individual elements. Python offers both positive and negative indexing, providing flexibility in how you retrieve data. Here's a breakdown:

Positive Indexing:

- Concept:**

- Positive indexing starts from the beginning of the list.
- The first element has an index of 0, the second has an index of 1, and so on.

- Example:**

- Let's say you have a list: `my_list = ['apple', 'banana', 'cherry']`
- `my_list[0]` would give you 'apple'.
- `my_list[1]` would give you 'banana'.
- `my_list[2]` would give you 'cherry'.

Negative Indexing:

- Concept:**

- Negative indexing starts from the end of the list.
- The last element has an index of -1, the second-to-last has an index of -2, and so on.

- **Example:**

- Using the same list: `my_list = ['apple', 'banana', 'cherry']`
- `my_list[-1]` would give you 'cherry'.
- `my_list[-2]` would give you 'banana'.
- `my_list[-3]` would give you 'apple'.

Key Points:

- **Zero-based:** Remember that Python uses zero-based indexing.

- **Use Cases:**

- Positive indexing is typically used when you want to access elements from the beginning of the list.
- Negative indexing is particularly useful when you want to access elements from the end of the list, especially when you don't know the list's length.

- **Error Handling:**

- If you try to access an index that is out of range (either positive or negative), Python will raise an `IndexError`.

Q3. Slicing a list: accessing a range of elements.

List slicing in Python is a powerful technique that allows you to extract a portion of a list, creating a new sublist. Here's a breakdown of how it works:

Basic Syntax:

- `list[start:stop:step]`
 - **start:** The index where the slice begins (inclusive). If omitted, it defaults to 0.
 - **stop:** The index where the slice ends (exclusive). If omitted, it defaults to the end of the list.
 - **step:** The interval between elements in the slice. If omitted, it defaults to 1.

Examples:

- **Getting a sublist:**
 - `my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`
 - `my_list[2:5]` will return `[2, 3, 4]`
(elements from index 2 up to, but not including, index 5).
- **Slicing from the beginning:**
 - `my_list[:4]` will return `[0, 1, 2, 3]`
(elements from the beginning up to, but not including, index 4).
- **Slicing to the end:**
 - `my_list[6:]` will return `[6, 7, 8, 9]`
(elements from index 6 to the end of the list).
- **Slicing with a step:**
 - `my_list[1:8:2]` will return `[1, 3, 5, 7]`
(elements from index 1 to 8, taking every second element).
- **Slicing with negative indexes:**

- `my_list[-3:]` will return `[7, 8, 9]` (the last three elements).
- `my_list[1:-1]` will return `[1,2,3,4,5,6,7,8]` (All elements except the first and last).
- **Reversing a list:**
 - `my_list[::-1]` will return `[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]` (the list in reverse order).

Key Points:

- Slicing creates a *new* list. It does not modify the original list.
- If the start or stop indexes are out of range, Python handles it gracefully.
- Slicing is very usefull for data manipulation.

Lab:

Q1. Write a Python program to create a list with elements of multiple data types (integers, strings, floats, etc.).

Python

```
def create_mixed_list():  
    """Creates a list with elements of various  
    data types."""
```

```
    mixed_list = [10, "hello", 3.14, True, [1, 2,  
3], {"name": "Alice"}, (4, 5)]  
    return mixed_list
```

```
# Example usage:
```

```
my_list = create_mixed_list()  
print(my_list)
```

```
# Accessing elements of different types:
```

```
print(f"Integer: {my_list[0]}")  
print(f"String: {my_list[1]}")  
print(f"Float: {my_list[2]}")  
print(f"Boolean: {my_list[3]}")  
print(f"Nested List: {my_list[4]}")  
print(f"Dictionary: {my_list[5]}")
```

```
print(f"Tuple: {my_list[6]}")
```

#Demonstrating accessing nested elements

```
print(f"Nested list element:  
{my_list[4][1]}") #Accessing the second  
element of the nested list
```

```
print(f"Dictionary value:  
{my_list[5]['name']}") #Accessing the  
value of the 'name' key in the dictionary.
```

Q2. Write a Python program to access elements at different index positions.

```
def access_list_elements(my_list,  
indices): """ Accesses and prints  
elements at specified index positions in a  
list. Args: my_list: The list to access  
elements from. indices: A list of integer  
indices to access. """  
for index in indices:  
    try: element = my_list[index]  
        print(f"Element at index {index}:  
{element}")  
    except IndexError:  
        print(f"Index {index} is out of range for the
```

```
list.") except TypeError: print("Index must
be an integer.") # Example usage: my_list
= [10, 20, 30, 40, 50] indices_to_access =
[0, 2, 4, -1, -3, 5, "a"] # Include valid and
invalid indices
access_list_elements(my_list,
indices_to_access) #Another Example
with a string. my_string = "Hello"
string_indices = [0,1,2,-1,5]
access_list_elements(my_string,
string_indices)
```

Practical Examples:

Q1. Write a Python program to create a list of multiple data type elements.

```
def create_mixed_data_list(): """Creates a
list containing elements of various data
types.""" mixed_list = [ 10, # Integer
"Python", # String 3.14, # Float True, #
Boolean [1, 2, 3], # Nested list {"name":
"John", "age": 30}, # Dictionary (5, 6), #
Tuple None, # NoneType 2 + 3j, # Complex
number b'bytes' # Bytes ] return
```

```
mixed_list # Example usage: my_list =  
create_mixed_data_list() print(my_list) #  
Demonstrating accessing individual  
elements and their types:  
print("\nAccessing individual elements  
and their types:") for i, element in  
enumerate(my_list): print(f"Element at  
index {i}: {element} (Type:  
{type(element)})") # Accessing nested list  
element print(f"\nNested list element at  
index 4, index 1: {my_list[4][1]}")  
#Accessing Dictionary element  
print(f"Dictionary value at key 'name'  
index 5: {my_list[5]['name']}")
```

Q2. Write a Python program to find the length of a list using the len() function.

```
def find_list_length(my_list): """ Finds and  
returns the length of a list using the len()  
function. Args: my_list: The list whose  
length needs to be found. Returns: The  
length of the list (an integer). """  
list_length = len(my_list) return list_length  
# Example Usage my_list1 = [1, 2, 3, 4, 5]
```

```
length1 = find_list_length(my_list1)
print(f"The length of my_list1 is:
{length1}") my_list2 = ["apple", "banana",
"cherry"] length2 =
find_list_length(my_list2) print(f"The
length of my_list2 is: {length2}") my_list3 =
[] #empty list length3 =
find_list_length(my_list3) print(f"The
length of my_list3 is: {length3}") my_list4 =
[1, "hello", 3.14, True, [1,2]] #Mixed data
types length4 = find_list_length(my_list4)
print(f"The length of my_list4 is:
{length4}")
```

2. List Operations

Theory:

Q1. Common list operations:

concatenation, repetition, membership.

When working with lists in Python, several core operations are frequently used.

These include concatenation, repetition,

and membership testing. Here's a breakdown of each:

1. Concatenation:

- Concept:**

- Concatenation combines two or more lists into a single new list.
- The + operator is used for list concatenation.

- Example:**

Python

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
combined_list = list1 + list2
```

```
print(combined_list) # Output: [1, 2, 3, 4, 5, 6]
```

2. Repetition:

- Concept:**

- Repetition creates a new list by repeating the elements of an

existing list a specified number of times.

- The * operator is used for list repetition.

. **Example:**

Python

```
my_list = ["a", "b"]
```

```
repeated_list = my_list * 3
```

```
print(repeated_list) # Output: ['a', 'b', 'a',  
'b', 'a', 'b']
```

3. Membership:

. **Concept:**

- Membership testing checks whether a specific element is present in a list.
- The in and not in operators are used for membership testing.

. **Example:**

Python

```
fruits = ["apple", "banana", "cherry"]
```

```
print("banana" in fruits) # Output: True  
print("orange" in fruits) # Output: False  
print("orange" not in fruits) # Output: True
```

Key takeaways:

- . These operations create new lists, they do not modify the original lists, unless the result of the operation is reassigned to the original list variable.**
- . in and not in are very useful for conditional statements, to control program flow.**
- . These operations are very common, and useful for many different programming tasks.**

Q2. Understanding list methods like append(), insert(), remove(), pop().

List methods in Python are essential for manipulating and modifying lists. Here's a breakdown of append(), insert(), remove(), and pop():

1. append():

- **Purpose:**

- Adds an element to the *end* of the list.

- **Syntax:**

- `list.append(element)`

- **Example:**

Python

```
my_list = [1, 2, 3]
```

```
my_list.append(4)
```

```
print(my_list) # Output: [1, 2, 3, 4]
```

2. `insert()`:

- **Purpose:**

- Inserts an element at a *specific index* in the list.

- **Syntax:**

- `list.insert(index, element)`

- **Example:**

Python

```
my_list = [1, 2, 3]
```

```
my_list.insert(1, 10) # Insert 10 at index  
1
```

```
print(my_list) # Output: [1, 10, 2, 3]
```

3. remove():

- **Purpose:**

- Removes the *first occurrence* of a specified element from the list.

- **Syntax:**

- list.remove(element)

- **Example:**

Python

```
my_list = [1, 2, 3, 2]
```

```
my_list.remove(2) # Removes the first  
'2'
```

```
print(my_list) # Output: [1, 3, 2]
```

- **Important:**

- If the element is not found, it raises a `ValueError`.

4. pop():

- **Purpose:**

- Removes and returns the element at a *specified index* (or the last element if no index is given).

- **Syntax:**

- `list.pop(index)` or `list.pop()`

- **Example:**

Python

```
my_list = [1, 2, 3]
```

```
removed_element = my_list.pop(1) #
```

Remove element at index 1

```
print(my_list) # Output: [1, 3]
```

```
print(removed_element) # Output: 2
```

```
my_list2 = [1,2,3]
```

```
last_element = my_list2.pop() #Removes  
the last element.
```

```
print(my_list2) #output: [1,2]
```

```
print(last_element) #output: 3
```

- **Important:**

- If the index is out of range, it raises an `IndexError`.
- If no index is provided, the last element of the list is removed and returned.

Key Differences:

- `append()`: Adds to the end.
- `insert()`: Adds at a specific position.
- `remove()`: Removes by value.
- `pop()`: Removes by index (and returns the removed value).

Lab:

Q1. Write a Python program to add elements to a list using `insert()` and `append()`.

```
def add_elements_to_list(): """Adds
elements to a list using insert() and
append().""" my_list = [1, 2, 3] # Append
elements to the end of the list
my_list.append(4) my_list.append(5)
```

```
print(f"List after appending 4 and 5:
{my_list}") # Insert an element at a
specific index my_list.insert(0, 0) # Insert
0 at the beginning (index 0) print(f"List
after inserting 0 at index 0: {my_list}")
my_list.insert(3, 10) #Insert 10 at index 3.
print(f"List after inserting 10 at index 3:
{my_list}") return my_list # Example
usage: result_list = add_elements_to_list()
#Demonstrating the returned list.
print(f"Final list: {result_list}")
```

Q2. Write a Python program to remove elements from a list using pop() and remove().

```
def remove_elements_from_list():
    """Removes elements from a list using
    pop() and remove().""" my_list = [10, 20,
    30, 40, 50, 30] # List with a duplicate
    element # Remove an element using pop()
    (by index) removed_element_pop =
    my_list.pop(2) # Remove element at index
    2 (30) print(f"List after pop(2): {my_list}")
    print(f"Removed element (pop):
```

```
{removed_element_pop}") # Remove an
element using remove() (by value)
my_list.remove(40) # Remove the first
occurrence of 40 print(f"List after
remove(40): {my_list}") #Remove the first
instance of a duplicate value.
my_list.remove(30) print(f"List after
remove(30): {my_list}") #Demonstrating
pop() without an index. removed_last =
my_list.pop() print(f"List after pop():
{my_list}") print(f"Removed last element
(pop): {removed_last}") return my_list #
Example usage: result_list =
remove_elements_from_list() print(f"Final
list: {result_list}") #Example of error
handling. try: my_list = [1,2,3]
my_list.remove(4) #4 does not exist.
except ValueError as e: print(f"Error: {e}")
try: my_list = [1,2,3] my_list.pop(5) #index
5 is out of range. except IndexError as e:
print(f"Error: {e}")
```

Practical Examples:

Q1. Write a Python program to update a list using insert() and append()

```
def update_list(original_list, insert_index,
insert_value, append_value): """ Updates
a list by inserting an element at a
specified index and appending an element
to the end. Args: original_list: The list to
be updated. insert_index: The index
where the element should be inserted.
insert_value: The value to be inserted.
append_value: The value to be appended.
Returns: The updated list. """ updated_list
= original_list[:] # Create a copy to avoid
modifying the original
updated_list.insert(insert_index,
insert_value)
updated_list.append(append_value)
return updated_list # Example usage:
my_list = [1, 2, 3, 4, 5] new_list =
update_list(my_list, 2, 100, 200)
print(f"Original list: {my_list}")
print(f"Updated list: {new_list}") #Another
example. another_list = ["apple",
```

```
"banana", "cherry"] another_updated_list
= update_list(another_list, 0, "mango",
"grape") print(f"Original list:
{another_list}") print(f"Updated list:
{another_updated_list}") #Example of
inserting at negative index.
negative_index_list = [1,2,3]
negative_index_updated =
update_list(negative_index_list, -1, 100,
200) print(f"Original list:
{negative_index_list}") print(f"Updated
list: {negative_index_updated}")
```

Q2. Write a Python program to remove elements from a list using pop() and remove().

```
def remove_elements_from_list(my_list,
pop_index=None, remove_value=None):
""" Removes elements from a list using
pop() and remove(). Args: my_list: The list
to modify. pop_index: The index to pop
(default: None). remove_value: The value
to remove (default: None). Returns: The
modified list. """ if pop_index is not None:
```

```
try: removed_element =
my_list.pop(pop_index) print(f"Popped
element at index {pop_index}:
{removed_element}") except IndexError:
print(f"Index {pop_index} is out of range.")
if remove_value is not None: try:
my_list.remove(remove_value)
print(f"Removed first occurrence of value:
{remove_value}") except ValueError:
print(f"Value {remove_value} not found in
the list.") return my_list # Example usage:
my_list = [10, 20, 30, 40, 50, 30] # Remove
using pop() my_list =
remove_elements_from_list(my_list,
pop_index=2) print(f"List after pop(2):
{my_list}") # Remove using remove()
my_list =
remove_elements_from_list(my_list,
remove_value=40) print(f"List after
remove(40): {my_list}") # Remove using
pop() without index my_list =
remove_elements_from_list(my_list,
pop_index=-1) print(f"List after pop():
```

```
{my_list}") # Remove a duplicate value  
my_list =  
remove_elements_from_list(my_list,  
remove_value=30) print(f"List after  
remove(30): {my_list}") #Example of error  
handling. my_list = [1,2,3]  
remove_elements_from_list(my_list,  
pop_index = 5)  
remove_elements_from_list(my_list,  
remove_value = 4)
```

3. Working with Lists

Theory:

Q1. Iterating over a list using loops.

Iterating over a list using loops is a fundamental concept in Python programming. Here's a breakdown of the common methods and how they work:

1. Using a for loop (Iterating by element):

- . This is the most straightforward way to iterate through each element in a list.

- The for loop directly accesses each element in the list, one at a time.

Python

```
my_list = ["apple", "banana", "cherry"]
```

```
for fruit in my_list:  
    print(fruit)
```

2. Using a for loop with range() and indices (Iterating by index):

- This method allows you to access elements using their indices.
- The range(len(my_list)) function generates a sequence of numbers from 0 to the length of the list minus 1.
- You can then use these indices to access the corresponding elements.

Python

```
my_list = ["apple", "banana", "cherry"]
```

```
for i in range(len(my_list)):
```

```
print(f"Index {i}: {my_list[i]}")
```

3. Using enumerate() (Iterating by index and element):

- . The enumerate() function provides both the index and the element during iteration.
- . This is often the most convenient way to iterate when you need both the index and the value.

Python

```
my_list = ["apple", "banana", "cherry"]
```

```
for index, fruit in enumerate(my_list):  
    print(f"Index {index}: {fruit}")
```

4. Using a while loop (Less common, but possible):

- . While less common for simple list iteration, while loops can be used.
- . You need to manually manage the index and the loop condition.

Python

```
my_list = ["apple", "banana", "cherry"]
```

```
i = 0
```

```
while i < len(my_list):
```

```
    print(my_list[i])
```

```
    i += 1
```

Key Considerations:

- **for loops:** Generally preferred for simple iteration over all elements.
- **range() and indices:** Useful when you need to access elements based on their positions.
- **enumerate():** Ideal when you need both indices and elements.
- **while loops:** Can be useful in more complex scenarios, but often less readable than for loops for basic list iteration.

- **Modifying Lists During Iteration:** Be cautious when modifying a list while iterating over it. It can lead to unexpected behavior. If you must modify a list while iterating, it's often safer to create a new list or use a copy of the original list.

Q2. Sorting and reversing a list using `sort()`, `sorted()`, and `reverse()`.

Python provides several built-in functions and methods for sorting and reversing lists. Here's a breakdown of `sort()`, `sorted()`, and `reverse()`:

1. `sort()` method:

- **Purpose:**
 - Sorts the list *in-place*, meaning it modifies the original list.
- **Syntax:**
 - `list.sort(key=..., reverse=...)`
- **Parameters:**

- **key:** An optional function that specifies a sorting key.
- **reverse:** An optional boolean value (True or False) that specifies whether to sort in descending order (True) or ascending order (False).

. Example:

Python

```
my_list = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
my_list.sort() # Sorts in ascending  
order
```

```
print(my_list) # Output: [1, 1, 2, 3, 4, 5,  
6, 9]
```

```
my_list.sort(reverse=True) #Sorts in  
descending order.
```

```
print(my_list) #output: [9, 6, 5, 4, 3, 2, 1,  
1]
```

```
string_list = ["apple", "banana",  
"Cherry"]
```

```
string_list.sort(key=str.lower) #Sorts  
ignoring case.
```

```
print(string_list) #output: ['apple',  
'banana', 'Cherry']
```

2. sorted() function:

- **Purpose:**

- Returns a *new* sorted list from the iterable. The original list remains unchanged.

- **Syntax:**

- `sorted(iterable, key=..., reverse=...)`

- **Parameters:**

- **iterable:** The iterable (e.g., list, tuple) to be sorted.
- **key:** An optional function that specifies a sorting key.

- **reverse:** An optional boolean value that specifies whether to sort in descending order.

- . Example:**

Python

```
my_list = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
sorted_list = sorted(my_list) # Creates a  
new sorted list
```

```
print(sorted_list) # Output: [1, 1, 2, 3, 4,  
5, 6, 9]
```

```
print(my_list) #output: [3, 1, 4, 1, 5, 9, 2,  
6] (original list unchanged)
```

```
reversed_sorted = sorted(my_list,  
reverse=True)
```

```
print(reversed_sorted) #output: [9, 6, 5,  
4, 3, 2, 1, 1]
```

3. reverse() method:

- . Purpose:**

- Reverses the elements of the list *in-place*.
- **Syntax:**
 - `list.reverse()`
- **Example:**

Python

```
my_list = [1, 2, 3, 4, 5]
```

```
my_list.reverse()
```

```
print(my_list) # Output: [5, 4, 3, 2, 1]
```

Key Differences:

- **`sort()` vs. `sorted()`:**
 - `sort()` modifies the original list, `sorted()` creates a new sorted list.
- **`reverse()`:**
 - Reverses the order of elements in the list *in-place*.
- `sorted()` can operate on any iterable, such as strings and tuples, while `sort()` and `reverse()` are list methods.

Q3. Basic list manipulations: addition, deletion, updating, and slicing.

Let's break down the fundamental list manipulations in Python: addition, deletion, updating, and slicing.

1. Addition (Concatenation and Appending):

- . Concatenation (+): Combines two lists into a new list.**

Python

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
combined_list = list1 + list2
```

```
print(combined_list) # Output: [1, 2, 3, 4, 5, 6]
```

- . Appending (append()): Adds a single element to the end of a list (modifies the original list).**

Python

```
my_list = [1, 2, 3]
```

```
my_list.append(4)
```

```
print(my_list) # Output: [1, 2, 3, 4]
```

- **Extending (extend()):** Adds multiple elements from another iterable (like another list) to the end of a list (modifies the original list).

Python

```
my_list = [1, 2, 3]
```

```
my_list.extend([4, 5, 6])
```

```
print(my_list) #output: [1, 2, 3, 4, 5, 6]
```

- **Inserting (insert()):** Inserts an element at a specific index.

Python

```
my_list = [1, 2, 3]
```

```
my_list.insert(1, 10) # Insert 10 at index 1
```

```
print(my_list) # Output: [1, 10, 2, 3]
```

2. Deletion:

- **del statement:** Deletes an element at a specific index or a slice of elements.

Python

```
my_list = [1, 2, 3, 4, 5]
del my_list[2] # Delete element at index 2
print(my_list) # Output: [1, 2, 4, 5]
```

```
del my_list[1:3] #delete a slice.
print(my_list) #output: [1,5]
```

- **remove() method:** Removes the first occurrence of a specific value.

Python

```
my_list = [1, 2, 3, 2]
my_list.remove(2) # Removes the first '2'
print(my_list) # Output: [1, 3, 2]
```

- **pop() method:** Removes and returns the element at a specific index (or the last element if no index is given).

Python

```
my_list = [1, 2, 3]
removed_element = my_list.pop(1) #
Remove element at index 1
```

`print(my_list)` # Output: [1, 3]

`print(removed_element)` # Output: 2

- **clear() method:** Removes all elements from the list.

Python

```
my_list = [1, 2, 3]
```

```
my_list.clear()
```

```
print(my_list) #output: []
```

3. Updating:

- **Direct assignment:** Modifies an element at a specific index.

Python

```
my_list = [1, 2, 3]
```

```
my_list[1] = 10 # Update element at index  
1
```

```
print(my_list) # Output: [1, 10, 3]
```

- **Slice assignment:** Modifies a range of elements.

Python

```
my_list = [1, 2, 3, 4, 5]
```

```
my_list[1:4] = [10, 20, 30] # Update  
elements from index 1 to 3
```

```
print(my_list) # Output: [1, 10, 20, 30, 5]
```

4. Slicing:

- . Creating sublists: Extracts a portion of a list.**

Python

```
my_list = [1, 2, 3, 4, 5]
```

```
sublist = my_list[1:4] # Elements from  
index 1 to 3
```

```
print(sublist) # Output: [2, 3, 4]
```

```
first_three = my_list[:3] #first three  
elements.
```

```
print(first_three) #output: [1, 2, 3]
```

```
last_two = my_list[-2:] #last two elements.
```

```
print(last_two) #output: [4,5]
```

```
every_other = my_list[::2] #every other element.
```

```
print(every_other) #output: [1, 3, 5]
```

```
reversed_list = my_list[::-1] #reverse the list.
```

```
print(reversed_list) #output: [5, 4, 3, 2, 1]
```

Lab:

Q1. Write a Python program to iterate over a list using a for loop

```
def iterate_list_for_loop(my_list): """  
Iterates over a list using a for loop and  
prints each element. Args: my_list: The list  
to iterate over. """ print("Iterating over  
the list using a for loop:") for item in  
my_list: print(item) # Example usage:  
my_list1 = [1, 2, 3, 4, 5]  
iterate_list_for_loop(my_list1) my_list2 =  
["apple", "banana", "cherry"]  
iterate_list_for_loop(my_list2) my_list3 =
```

[10, "hello", 3.14, True]

**iterate_list_for_loop(my_list3) my_list4 = []
#empty list iterate_list_for_loop(my_list4)**

Q2. Write a Python program to sort a list using both sort() and sorted().

```
def sort_list_methods(my_list): """ Sorts a  
list using both sort() and sorted() and  
prints the results. Args: my_list: The list to  
sort. """ print(f"Original list: {my_list}") #  
Using sort() (in-place sorting) list_sort =  
my_list[:] # Create a copy to avoid  
modifying the original list list_sort.sort()  
print(f"List after sort(): {list_sort}")  
print(f"Original list after sort() call:  
{my_list}") #Demonstrates original list is  
unchanged. # Using sorted() (returns a  
new sorted list) list_sorted =  
sorted(my_list) print(f"List after sorted():  
{list_sorted}") print(f"Original list after  
sorted() call: {my_list}") #Demonstrates  
original list is unchanged. #Using sorted()  
with reverse reversed_sorted =  
sorted(my_list, reverse = True) print(f"List
```

```
after sorted(reverse = True):
{reversed_sorted}") #Using sort() with
reverse list_sort_reversed = my_list[:]
list_sort_reversed.sort(reverse = True)
print(f"List after sort(reverse = True):
{list_sort_reversed}") # Example usage:
my_list = [5, 2, 8, 1, 9, 4]
sort_list_methods(my_list) my_string_list =
["banana", "apple", "Cherry", "date"]
sort_list_methods(my_string_list)
```

Practical Examples:

Q1. Write a Python program to iterate through a list and print each element

```
def print_list_elements(my_list): """
Iterates through a list and prints each
element. Args: my_list: The list to iterate
through. """ if not my_list: # Check if the
list is empty print("The list is empty.")
return print("Elements of the list:") for
element in my_list: print(element) #
Example Usage: list1 = [1, 2, 3, 4, 5]
print_list_elements(list1) list2 = ["apple",
```

```
"banana", "cherry", "date"]
print_list_elements(list2) list3 = [10,
"hello", 3.14, True, [1, 2]]
print_list_elements(list3) empty_list = []
print_list_elements(empty_list)
```

Q2. Write a Python program to insert elements into an empty list using a for loop and append().

```
def
insert_elements_into_empty_list(elements
): """ Inserts elements into an empty list
using a for loop and append(). Args:
elements: An iterable (e.g., list, tuple,
range) containing the elements to insert.
Returns: The list with inserted elements.
""" my_list = [] # Create an empty list for
element in elements:
my_list.append(element) # Append each
element to the list return my_list #
Example usage: elements_to_insert = [1,
2, 3, 4, 5] filled_list =
insert_elements_into_empty_list(elements
_to_insert) print(f"List after inserting
```

```
elements: {filled_list}")
elements_to_insert2 = range(10, 20, 2)
#Using range. filled_list2 =
insert_elements_into_empty_list(elements
_to_insert2) print(f"List after inserting
elements from range: {filled_list2}")
elements_to_insert3 = ("apple", "banana",
"cherry") #using tuple. filled_list3 =
insert_elements_into_empty_list(elements
_to_insert3) print(f"List after inserting
tuple elements: {filled_list3}")
elements_to_insert4 = [] #empty list.
filled_list4 =
insert_elements_into_empty_list(elements
_to_insert4) print(f"List after inserting
empty list elements: {filled_list4}")
```

4. Tuple

Theory:

Q1. Introduction to tuples, immutability.

Tuples in Python are a sequence data type, similar to lists, but with a crucial difference: they are **immutable**. This

immutability has significant implications for how you use them. Let's delve into the details:

What are Tuples?

- Tuples are ordered collections of items.
- They can hold elements of various data types (integers, strings, floats, other tuples, etc.).
- Tuples are defined by enclosing elements within parentheses (), separated by commas.

Creating Tuples:

Python

`my_tuple = (1, 2, 3, "hello", 3.14)` # A tuple with mixed data types

`empty_tuple = ()` # An empty tuple

`single_item_tuple = (5,)` # A tuple with a single item (note the trailing comma)

Immutability:

- **Immutability** means that once a tuple is created, its elements *cannot be changed*. You cannot:
 - Add new elements.
 - Remove existing elements.
 - Modify the values of existing elements.
- Attempting to do any of these operations will result in a `TypeError`.

Why Immutability?

- **Data Integrity:** Immutability ensures that the data in a tuple remains constant throughout the program. This is useful for storing data that should not be modified, such as configuration settings or coordinates.
- **Performance:** Tuples are generally more memory-efficient than lists because their size is fixed.
- **Use as Dictionary Keys:** Tuples can be used as keys in dictionaries, whereas

lists cannot. This is because dictionary keys must be immutable.

- **Safer Code:** Immutability can prevent accidental modifications to data, making your code more robust.

Accessing Tuple Elements:

- You access tuple elements using indexing, just like with lists.

Python

```
my_tuple = (10, 20, 30)
```

```
print(my_tuple[0]) # Output: 10
```

```
print(my_tuple[1]) # Output: 20
```

```
print(my_tuple[-1]) #output: 30
```

- Slicing also works with tuples.

Python

```
my_tuple = (10, 20, 30, 40, 50)
```

```
print(my_tuple[1:4]) #output: (20, 30, 40)
```

Tuple Operations:

- Tuples support operations like concatenation (+) and repetition (*), which create *new* tuples.

Python

```
tuple1 = (1, 2, 3)
```

```
tuple2 = (4, 5, 6)
```

```
combined_tuple = tuple1 + tuple2 #
```

```
Output: (1, 2, 3, 4, 5, 6)
```

```
repeated_tuple = tuple1 * 2 # Output: (1,  
2, 3, 1, 2, 3)
```

- The in and not in operators work with tuples.
- The len() function works with tuples.

When to Use Tuples vs. Lists:

- Use tuples when you need an ordered collection of items that should not be changed.
- Use lists when you need a collection that can be modified.

Q2. Creating and accessing elements in a tuple.

Creating and accessing elements in a tuple is straightforward, similar to lists, but with the key difference that tuples are immutable (cannot be changed after creation). Here's a breakdown:

1. Creating Tuples:

- . Using Parentheses ():**

- The most common way is to enclose elements within parentheses, separated by commas.
- Example:

Python

```
my_tuple = (1, 2, "hello", 3.14, True)
```

- . Empty Tuple:**

- An empty tuple is created using empty parentheses.

Python

```
empty_tuple = ()
```

- **Single-Element Tuple:**

- A tuple with a single element requires a trailing comma.

Python

```
single_tuple = (5,) # The comma is essential!
```

- **Tuple Packing (Without Parentheses):**

- In some cases, you can create a tuple without parentheses (tuple packing).

Python

```
my_tuple = 1, 2, "test" # Tuple packing
```

- **Using the tuple() Constructor:**

- You can create a tuple from an iterable (like a list or string) using the tuple() constructor.

Python

```
my_list = [1, 2, 3]
```

```
my_tuple = tuple(my_list)
```

2. Accessing Elements:

- . Indexing:**

- Tuple elements are accessed using their index, starting from 0 for the first element.
- Example:

Python

```
my_tuple = (10, 20, 30, 40)
```

```
print(my_tuple[0]) # Output: 10
```

```
print(my_tuple[2]) # Output: 30
```

- . Negative Indexing:**

- Negative indices access elements from the end of the tuple.
- -1 refers to the last element, -2 to the second-to-last, and so on.
- Example:

Python

```
my_tuple = (10, 20, 30, 40)
```

```
print(my_tuple[-1]) # Output: 40
```

print(my_tuple[-3]) # Output: 20

- . Slicing:**

- Slicing extracts a portion of the tuple into a new tuple.
- Syntax: tuple[start:stop:step]
- Example:

Python

my_tuple = (10, 20, 30, 40, 50)

print(my_tuple[1:4]) # Output: (20, 30, 40)

print(my_tuple[:3]) #output: (10,20,30)

print(my_tuple[2:]) #output: (30,40,50)

print(my_tuple[::2]) #output: (10,30,50)

- . Unpacking:**

- Tuple unpacking assigns tuple elements to variables.
- Example:

Python

my_tuple = (100, 200, 300)

a, b, c = my_tuple

print(a) # Output: 100

print(b) # Output: 200

print(c) # Output: 300

**Q3. Basic operations with tuples:
concatenation, repetition, membership.**

**Tuples, being immutable sequences,
support a specific set of basic operations.
Here's a breakdown of concatenation,
repetition, and membership:**

1. Concatenation:

- . Concept:**

- Concatenation combines two or more tuples into a new tuple.
- The + operator is used for tuple concatenation.

- . Example:**

Python

tuple1 = (1, 2, 3)

tuple2 = ("a", "b", "c")

combined_tuple = tuple1 + tuple2

```
print(combined_tuple) # Output: (1, 2, 3, 'a', 'b', 'c')
```

- **Important:** Concatenation creates a *new* tuple. The original tuples remain unchanged.

2. Repetition:

- **Concept:**
 - Repetition creates a new tuple by repeating the elements of an existing tuple a specified number of times.
 - The * operator is used for tuple repetition.
- **Example:**

Python

```
my_tuple = ("hello", 2)
repeated_tuple = my_tuple * 3
print(repeated_tuple) # Output: ('hello', 2, 'hello', 2, 'hello', 2)
```

- **Important:** Repetition creates a *new* tuple. The original tuple remains unchanged.

3. Membership:

- **Concept:**
 - Membership testing checks whether a specific element is present in a tuple.
 - The `in` and `not in` operators are used for membership testing.
- **Example:**

Python

```
my_tuple = (10, 20, 30, "python")  
print(20 in my_tuple)    # Output: True  
print("java" in my_tuple) # Output: False  
print("java" not in my_tuple) #Output:
```

True

Lab:

Q1. Write a Python program to create a tuple with multiple data types

```

def create_mixed_tuple(): """Creates a
tuple with elements of multiple data
types.""" mixed_tuple = (10, "hello", 3.14,
True, [1, 2, 3], {"name": "Alice"}, (4, 5))
return mixed_tuple # Example usage:
my_tuple = create_mixed_tuple()
print(my_tuple) # Accessing elements of
different types: print(f"Integer:
{my_tuple[0]}") print(f"String:
{my_tuple[1]}") print(f"Float:
{my_tuple[2]}") print(f"Boolean:
{my_tuple[3]}") print(f"Nested List:
{my_tuple[4]}") print(f"Dictionary:
{my_tuple[5]}") print(f"Tuple:
{my_tuple[6]}") #Demonstrating accessing
nested elements print(f"Nested list
element: {my_tuple[4][1]}") #Accessing
the second element of the nested list
print(f"Dictionary value:
{my_tuple[5]['name']}") #Accessing the
value of the 'name' key in the dictionary.

```

Q2. Write a Python program to concatenate two tuples.

```

def concatenate_tuples(tuple1, tuple2):
    """ Concatenates two tuples and returns
    the resulting tuple. Args: tuple1: The first
    tuple. tuple2: The second tuple. Returns:
    A new tuple containing the elements of
    both input tuples. """
    concatenated_tuple = tuple1 + tuple2
    return concatenated_tuple

# Example usage:
tuple_a = (1, 2, 3)
tuple_b = ("a", "b", "c")
result_tuple = concatenate_tuples(tuple_a, tuple_b)
print(f"Concatenated tuple: {result_tuple}")

tuple_c = (10, 20)
tuple_d = (30, 40, 50)
result_tuple2 = concatenate_tuples(tuple_c, tuple_d)
print(f"Concatenated tuple: {result_tuple2}")

tuple_e = ()
tuple_f = (1, 2, 3)
result_tuple3 = concatenate_tuples(tuple_e, tuple_f)
print(f"Concatenated tuple: {result_tuple3}")

tuple_g = ("test",)
tuple_h = ("another",)
result_tuple4 = concatenate_tuples(tuple_g, tuple_h)

```

```
print(f"Concatenated tuple:  
{result_tuple4}")
```

Practical Examples:

Q1. Write a Python program to convert a list into a tuple

```
def list_to_tuple(input_list): """ Converts a  
list into a tuple. Args: input_list: The list to  
convert. Returns: A tuple containing the  
same elements as the input list. """
```

```
result_tuple = tuple(input_list) return  
result_tuple # Example usage: my_list1 =  
[1, 2, 3, 4, 5] my_tuple1 =  
list_to_tuple(my_list1) print(f"List:  
{my_list1}") print(f"Tuple: {my_tuple1}")  
my_list2 = ["apple", "banana", "cherry"]  
my_tuple2 = list_to_tuple(my_list2)  
print(f"List: {my_list2}") print(f"Tuple:  
{my_tuple2}") my_list3 = [10, "hello", 3.14,  
True] my_tuple3 = list_to_tuple(my_list3)  
print(f"List: {my_list3}") print(f"Tuple:  
{my_tuple3}") my_list4 = [] #empty list  
my_tuple4 = list_to_tuple(my_list4)
```

```
print(f"List: {my_list4}") print(f"Tuple:  
{my_tuple4}")
```

Q2. Write a Python program to create a tuple with multiple data types.

```
def create_mixed_tuple(): """Creates a  
tuple with elements of multiple data  
types.""" mixed_tuple = (10, "hello", 3.14,  
True, [1, 2, 3], {"name": "Alice"}, (4, 5))  
return mixed_tuple # Example usage:  
my_tuple = create_mixed_tuple()  
print(my_tuple) # Accessing elements of  
different types: print(f"Integer:  
{my_tuple[0]}") print(f"String:  
{my_tuple[1]}") print(f"Float:  
{my_tuple[2]}") print(f"Boolean:  
{my_tuple[3]}") print(f"Nested List:  
{my_tuple[4]}") print(f"Dictionary:  
{my_tuple[5]}") print(f"Tuple:  
{my_tuple[6]}") #Demonstrating accessing  
nested elements print(f"Nested list  
element: {my_tuple[4][1]}") #Accessing  
the second element of the nested list  
print(f"Dictionary value:
```

`{my_tuple[5]['name']})` #Accessing the value of the 'name' key in the dictionary.

Q3. Write a Python program to concatenate two tuples into one.

```
def concatenate_tuples(tuple1, tuple2):  
    """ Concatenates two tuples into a single  
    tuple. Args: tuple1: The first tuple. tuple2:  
    The second tuple. Returns: A new tuple  
    containing all elements from both input  
    tuples. """ return tuple1 + tuple2 #
```

```
Example Usage: tuple_a = (1, 2, 3) tuple_b  
= ('a', 'b', 'c') result_tuple =  
concatenate_tuples(tuple_a, tuple_b)  
print(f"Concatenated tuple:  
{result_tuple}") tuple_c = (10, 20) tuple_d  
= (30, 40, 50) result_tuple2 =  
concatenate_tuples(tuple_c, tuple_d)  
print(f"Concatenated tuple:  
{result_tuple2}") tuple_e = () #empty tuple  
tuple_f = (1,2,3) result_tuple3 =  
concatenate_tuples(tuple_e, tuple_f)  
print(f"Concatenated tuple:  
{result_tuple3}") tuple_g = ("test",) #single
```

```
item tuple tuple_h = ("another",) #single
item tuple result_tuple4 =
concatenate_tuples(tuple_g, tuple_h)
print(f"Concatenated tuple:
{result_tuple4}")
```

Q4. Write a Python program to access the value of the first index in a tuple.

```
def get_first_tuple_element(input_tuple):
    """ Accesses and returns the value of the
    first index (0) in a tuple. Args: input_tuple:
    The tuple from which to retrieve the first
    element. Returns: The value of the first
    element in the tuple, or None if the tuple is
    empty. """ if input_tuple: # Check if the
    tuple is not empty return input_tuple[0]
    else: return None # Return None for an
    empty tuple # Example usage: my_tuple1 =
    (10, 20, 30) first_element1 =
    get_first_tuple_element(my_tuple1)
    print(f"First element of my_tuple1:
    {first_element1}") my_tuple2 = ("apple",
    "banana", "cherry") first_element2 =
    get_first_tuple_element(my_tuple2)
```

```
print(f"First element of my_tuple2:  
{first_element2}") my_tuple3 = (True,  
False) first_element3 =  
get_first_tuple_element(my_tuple3)  
print(f"First element of my_tuple3:  
{first_element3}") empty_tuple = ()  
first_element4 =  
get_first_tuple_element(empty_tuple)  
print(f"First element of empty_tuple:  
{first_element4}") single_tuple = (1,)   
first_element5 =  
get_first_tuple_element(single_tuple)  
print(f"First element of single_tuple:  
{first_element5}")
```

5. Accessing Tuples

Theory:

Q1. Accessing tuple elements using positive and negative indexing.

Accessing tuple elements using positive and negative indexing is very similar to accessing list elements. Here's a breakdown:

Positive Indexing:

- Concept:**

- Positive indexing starts from the beginning of the tuple.
- The first element has an index of 0, the second has an index of 1, and so on.

- Example:**

Python

```
my_tuple = ("apple", "banana",  
"cherry", "date")
```

```
print(my_tuple[0]) # Output: apple
```

```
print(my_tuple[1]) # Output: banana
```

```
print(my_tuple[3]) # Output: date
```

Negative Indexing:

- Concept:**

- Negative indexing starts from the end of the tuple.

- The last element has an index of -1, the second-to-last has an index of -2, and so on.

. Example:

Python

```
my_tuple = ("apple", "banana",  
"cherry", "date")
```

```
print(my_tuple[-1]) # Output: date
```

```
print(my_tuple[-2]) # Output: cherry
```

```
print(my_tuple[-4]) # Output: apple
```

Combined Example:

Python

```
my_tuple = (10, 20, 30, 40, 50)
```

```
print(my_tuple[0]) # Output: 10 (first  
element, positive index)
```

```
print(my_tuple[-1]) # Output: 50 (last  
element, negative index)
```

```
print(my_tuple[2]) # Output: 30
```

```
print(my_tuple[-3]) # Output: 30
```

Q2. Slicing a tuple to access ranges of elements.

Slicing a tuple in Python allows you to extract a portion of the tuple, creating a new tuple containing the selected elements. The syntax is very similar to slicing lists.

Basic Syntax:

Python

tuple[start:stop:step]

- . start: The index where the slice begins (inclusive). If omitted, it defaults to 0.**
- . stop: The index where the slice ends (exclusive). If omitted, it defaults to the end of the tuple.**
- . step: The interval between elements in the slice. If omitted, it defaults to 1.**

Examples:

Python

```
my_tuple = (10, 20, 30, 40, 50, 60, 70, 80,  
90)
```

Basic slicing:

```
print(my_tuple[2:5]) # Output: (30, 40, 50)
```

Slicing from the beginning:

```
print(my_tuple[:4]) # Output: (10, 20, 30,  
40)
```

Slicing to the end:

```
print(my_tuple[5:]) # Output: (60, 70, 80,  
90)
```

Slicing with a step:

```
print(my_tuple[1:8:2]) # Output: (20, 40,  
60, 80)
```

Slicing with negative indices:

```
print(my_tuple[-4:-1]) # Output: (60, 70, 80)
```

Slicing with a negative step (reversing):

```
print(my_tuple[::-1]) # Output: (90, 80, 70, 60, 50, 40, 30, 20, 10)
```

Slicing with only a step

```
print(my_tuple[::2]) #Output: (10, 30, 50, 70, 90)
```

#Slicing the whole tuple

```
print(my_tuple[:]) #Output: (10, 20, 30, 40, 50, 60, 70, 80, 90)
```

Lab:

Q1. Write a Python program to access values between index 1 and 5 in a tuple

```
def access_tuple_range(input_tuple): """  
Accesses and returns values between  
index 1 and 5 (exclusive) in a tuple. Args:
```

input_tuple: The tuple to access. Returns:
A new tuple containing values from index
1 to 4 (5 is exclusive), or an empty tuple if
the input tuple is too short. """ if
len(input_tuple) >= 6: # Check if the tuple
has enough elements return
input_tuple[1:5] # Slice from index 1 to 4
(5 is exclusive) else: return () # Return an
empty tuple if the input is too short #
Example usage: my_tuple1 = (10, 20, 30,
40, 50, 60, 70) result1 =
access_tuple_range(my_tuple1)
print(f"Values between index 1 and 5:
{result1}") my_tuple2 = ("a", "b", "c", "d",
"e") result2 =
access_tuple_range(my_tuple2)
print(f"Values between index 1 and 5:
{result2}") my_tuple3 = (1, 2, 3) # Tuple too
short result3 =
access_tuple_range(my_tuple3)
print(f"Values between index 1 and 5:
{result3}") my_tuple4 = (1,2,3,4,5) result4
= access_tuple_range(my_tuple4)

```
print(f"Values between index 1 and 5:  
{result4}") my_tuple5 = (1,2,3,4,5,6)  
result5 = access_tuple_range(my_tuple5)  
print(f"Values between index 1 and 5:  
{result5}")
```

Q2. Write a Python program to access alternate values between index 1 and 5 in a tuple.

```
def  
access_alterate_tuple_range(input_tuple  
): """ Accesses and returns alternate  
values between index 1 and 5 (exclusive)  
in a tuple. Args: input_tuple: The tuple to  
access. Returns: A new tuple containing  
alternate values from index 1 to 4 (5 is  
exclusive), or an empty tuple if the input  
tuple is too short. """ if len(input_tuple) >=  
6: # Check if the tuple has enough  
elements return input_tuple[1:5:2] # Slice  
with step 2 for alternate values else:  
return () # Return an empty tuple if the  
input is too short # Example usage:  
my_tuple1 = (10, 20, 30, 40, 50, 60, 70)
```

```
result1 =  
access_alternate_tuple_range(my_tuple1)  
print(f"Alternate values between index 1  
and 5: {result1}") my_tuple2 = ("a", "b",  
"c", "d", "e") result2 =  
access_alternate_tuple_range(my_tuple2)  
print(f"Alternate values between index 1  
and 5: {result2}") my_tuple3 = (1, 2, 3) #  
Tuple too short result3 =  
access_alternate_tuple_range(my_tuple3)  
print(f"Alternate values between index 1  
and 5: {result3}") my_tuple4 = (1,2,3,4,5)  
result4 =  
access_alternate_tuple_range(my_tuple4)  
print(f"Alternate values between index 1  
and 5: {result4}") my_tuple5 =  
(1,2,3,4,5,6,7) result5 =  
access_alternate_tuple_range(my_tuple5)  
print(f"Alternate values between index 1  
and 5: {result5}")
```

Practical Examples:

Q1. Write a Python program to access values between index 1 and 5 in a tuple.

```
def access_tuple_range(input_tuple): """
Accesses and returns values between
index 1 and 5 (exclusive) in a tuple. Args:
input_tuple: The tuple to access. Returns:
A new tuple containing values from index
1 to 4 (5 is exclusive), or an empty tuple if
the input tuple is too short. """ if
len(input_tuple) >= 5: # Check if the tuple
has at least 5 elements return
input_tuple[1:5] # Slice from index 1 to 4
(5 is exclusive) else: return () # Return an
empty tuple if the input is too short #
Example usage: my_tuple1 = (10, 20, 30,
40, 50, 60, 70) result1 =
access_tuple_range(my_tuple1)
print(f"Values between index 1 and 5:
{result1}") my_tuple2 = ("a", "b", "c", "d",
"e") result2 =
access_tuple_range(my_tuple2)
print(f"Values between index 1 and 5:
{result2}") my_tuple3 = (1, 2, 3) # Tuple too
short result3 =
access_tuple_range(my_tuple3)
```

```
print(f"Values between index 1 and 5:  
{result3}") my_tuple4 = (1, 2, 3, 4, 5)  
result4 = access_tuple_range(my_tuple4)  
print(f"Values between index 1 and 5:  
{result4}") my_tuple5 = (1,2,3,4) result5 =  
access_tuple_range(my_tuple5)  
print(f"Values between index 1 and 5:  
{result5}")
```

Q2. Write a Python program to access the value from the last index in a tuple.

```
def get_last_tuple_element(input_tuple):  
    """ Accesses and returns the value of the  
    last index in a tuple. Args: input_tuple:  
    The tuple to access. Returns: The value of  
    the last element in the tuple, or None if the  
    tuple is empty. """ if input_tuple: # Check  
    if the tuple is not empty return  
    input_tuple[-1] # Access the last element  
    using negative indexing else: return None  
# Return None for an empty tuple #  
Example usage: my_tuple1 = (10, 20, 30,  
40) last_element1 =  
get_last_tuple_element(my_tuple1)
```

```
print(f"Last element of my_tuple1:
{last_element1}") my_tuple2 = ("apple",
"banana", "cherry") last_element2 =
get_last_tuple_element(my_tuple2)
print(f"Last element of my_tuple2:
{last_element2}") empty_tuple = ()
last_element3 =
get_last_tuple_element(empty_tuple)
print(f"Last element of empty_tuple:
{last_element3}") single_tuple = (5,)
last_element4 =
get_last_tuple_element(single_tuple)
print(f"Last element of single_tuple:
{last_element4}") mixed_tuple = (1,"hello",
3.14, True) last_element5 =
get_last_tuple_element(mixed_tuple)
print(f"Last element of mixed_tuple:
{last_element5}")
```

6. Dictionaries

Theory:

Q1. Introduction to dictionaries: key-value pairs.

Dictionaries in Python are a powerful and versatile data structure that store data in key-value pairs. This means that each piece of data is associated with a unique identifier, or "key." Here's a breakdown of the key concepts:

Key-Value Pairs:

- . A dictionary is a collection of key-value pairs.**
- . Key:**
 - A unique identifier that is used to access the corresponding value.**
 - Keys must be immutable (e.g., strings, numbers, tuples).**
- . Value:**
 - The data associated with a key.**
 - Values can be of any data type (strings, numbers, lists, other dictionaries, etc.).**

Creating Dictionaries:

- . Dictionaries are created using curly braces {}.
- . Key-value pairs are separated by colons :.
- . Pairs are separated by commas ,.

Python

```
my_dict = {"name": "Alice", "age": 30,  
"city": "New York"}  
empty_dict = {}
```

Accessing Values:

- . Values are accessed using their keys within square brackets [].

Python

```
print(my_dict["name"]) # Output: Alice  
print(my_dict["age"]) # Output: 30
```

- . The get() method is also used to access values. It returns None (or a specified default value) if the key is not found, preventing KeyError.

Python

```
print(my_dict.get("city"))    # Output:  
New York
```

```
print(my_dict.get("country")) # Output:  
None
```

```
print(my_dict.get("country",  
"Unknown")) #Output: Unknown
```

Mutability:

- . Dictionaries are mutable, meaning you can add, modify, and remove key-value pairs after the dictionary is created.

Adding and Modifying Key-Value Pairs:

- . To add a new key-value pair, assign a value to a new key.

Python

```
my_dict["job"] = "Engineer"  
  
print(my_dict) #output: {'name': 'Alice',  
'age': 30, 'city': 'New York', 'job':  
'Engineer'}
```

- . To modify an existing value, assign a new value to the existing key.

Python

```
my_dict["age"] = 31  
  
print(my_dict) #output: {'name': 'Alice',  
'age': 31, 'city': 'New York', 'job':  
'Engineer'}
```

Removing Key-Value Pairs:

- . The del keyword removes a key-value pair.

Python

```
del my_dict["city"]  
  
print(my_dict) #output: {'name': 'Alice',  
'age': 31, 'job': 'Engineer'}
```

- . The pop() method removes a key-value pair and returns the value.

Python

```
job = my_dict.pop("job")  
  
print(my_dict) #output: {'name': 'Alice',  
'age': 31}  
  
print(job) #output: Engineer
```

- The `clear()` method removes all key-value pairs from the dictionary.

Python

```
my_dict.clear()
```

```
print(my_dict) #output: {}
```

Key Advantages:

- **Efficient Data Retrieval:** Dictionaries provide fast access to values based on their keys.
- **Flexibility:** Dictionaries can store data of various types and structures.
- **Readability:** Key-value pairs make code more readable and easier to understand.

Q2. Accessing, adding, updating, and deleting dictionary elements.

Dictionaries in Python are highly flexible, allowing you to easily access, add, update, and delete elements. Here's a detailed explanation:

1. Accessing Elements:

- **Using Square Brackets []:**

- Access a value by its key.
- If the key doesn't exist, it raises a **KeyError**.

Python

```
my_dict = {"name": "Alice", "age": 30,  
"city": "New York"}
```

```
print(my_dict["name"]) # Output: Alice
```

- **Using get() Method:**

- Access a value by its key.
- If the key doesn't exist, it returns **None** (or a specified default value).

Python

```
print(my_dict.get("age"))    # Output: 30
```

```
print(my_dict.get("country")) # Output:  
None
```

```
print(my_dict.get("country", "Unknown"))  
#Output: Unknown
```

2. Adding Elements:

- **Direct Assignment:**

- Assign a value to a new key.
- If the key already exists, it updates the value.

Python

```
my_dict["job"] = "Engineer"
```

```
print(my_dict) # Output: {'name': 'Alice',  
'age': 30, 'city': 'New York', 'job':  
'Engineer'}
```

3. Updating Elements:

- . Direct Assignment:**

- Assign a new value to an existing key.

Python

```
my_dict["age"] = 31
```

```
print(my_dict) # Output: {'name': 'Alice',  
'age': 31, 'city': 'New York', 'job':  
'Engineer'}
```

- . update() Method:**

- Updates the dictionary with elements from another dictionary or an iterable of key-value pairs.

Python

```
my_dict.update({"city": "London",  
"country": "UK"})  
  
print(my_dict) #output: {'name': 'Alice',  
'age': 31, 'city': 'London', 'job': 'Engineer',  
'country': 'UK'}
```

4. Deleting Elements:

- **del Statement:**

- Deletes a key-value pair.
- If the key doesn't exist, it raises a **KeyError**.

Python

```
del my_dict["job"]  
  
print(my_dict) # Output: {'name': 'Alice',  
'age': 31, 'city': 'London', 'country': 'UK'}
```

- **pop() Method:**

- Removes a key-value pair and returns the value.
- If the key doesn't exist, it raises a **KeyError** (or returns a specified default value).

Python

```
country = my_dict.pop("country")  
print(my_dict) #output: {'name': 'Alice',  
'age': 31, 'city': 'London'}  
print(country) #output: UK
```

```
city = my_dict.pop("city", "City not found")  
#If city does not exist, "city not found" is  
returned.
```

```
print(city) #output: London
```

• **popitem() Method:**

- Removes and returns an arbitrary key-value pair (usually the last one inserted in Python 3.7+).

- If the dictionary is empty, it raises a **KeyError**.

Python

```
item = my_dict.popitem()
```

```
print(my_dict) #output: {'name': 'Alice',  
'age': 31}
```

```
print(item) #output: ('city', 'London')
```

. clear() Method:

- Removes all key-value pairs from the dictionary.

Python

```
my_dict.clear()
```

```
print(my_dict) # Output: {}
```

Q3. Dictionary methods like keys(), values(), and items().

Python dictionaries provide several useful methods for accessing and working with their contents. Among the most common are keys(), values(), and items(). Here's a breakdown of each:

1. keys():

- . Purpose:**

- Returns a view object that displays a list of all the keys in the dictionary.
- This view object reflects any changes made to the dictionary.

- . Example:**

Python

```
my_dict = {"name": "Alice", "age": 30,  
"city": "New York"}
```

```
keys = my_dict.keys()
```

```
print(keys) # Output: dict_keys(['name',  
'age', 'city'])
```

```
my_dict["job"] = "Engineer"
```

```
print(keys) #output: dict_keys(['name',  
'age', 'city', 'job'])
```

2. values():

- . Purpose:**

- Returns a view object that displays a list of all the values in the dictionary.
- This view object reflects any changes made to the dictionary.

. **Example:**

Python

```
my_dict = {"name": "Alice", "age": 30,  
"city": "New York"}
```

```
values = my_dict.values()
```

```
print(values) # Output: dict_values([  
'Alice', 30, 'New York'])
```

```
my_dict["age"] = 31
```

```
print(values) #output:  
dict_values(['Alice', 31, 'New York'])
```

3. items():

. **Purpose:**

- Returns a view object that displays a list of all the key-value pairs in the dictionary as tuples.
- This view object reflects any changes made to the dictionary.

. Example:

Python

```
my_dict = {"name": "Alice", "age": 30,  
"city": "New York"}
```

```
items = my_dict.items()
```

```
print(items) # Output:
```

```
dict_items([('name', 'Alice'), ('age', 30),  
('city', 'New York')])
```

```
del my_dict["city"]
```

```
print(items) #output:
```

```
dict_items([('name', 'Alice'), ('age', 30)])
```

Key Points:

. View Objects:

- The objects returned by these methods are "view objects," meaning they provide a dynamic view of the dictionary. If the dictionary changes, the view objects reflect those changes.
- **Iteration:**
 - These view objects are iterable, so you can use them in for loops.

Python

```
my_dict = {"name": "Alice", "age": 30}
for key, value in my_dict.items():
    print(f"{key}: {value}")
```

- **Conversion to Lists:**
 - If you need a static list of keys, values, or items, you can convert the view objects to lists using the `list()` constructor.

Python

```
my_dict = {"name": "Alice", "age": 30}
```

```
keys_list = list(my_dict.keys())  
print(keys_list) # Output: ['name', 'age']
```

Lab:

Q1. Write a Python program to create a dictionary with 6 key-value pairs.

```
def create_six_item_dictionary():  
    """Creates a dictionary with 6 key-value  
    pairs.""" my_dict = { "name": "Alice",  
    "age": 30, "city": "New York", "job":  
    "Engineer", "country": "USA", "email":  
    "alice@example.com" } return my_dict #  
Example usage: six_item_dict =  
create_six_item_dictionary()  
print(six_item_dict) #Demonstrating  
accessing the dictionary. print(f"Name:  
{six_item_dict['name']}") print(f"Age:  
{six_item_dict['age']}") print(f"City:  
{six_item_dict['city']}") print(f"Job:  
{six_item_dict['job']}") print(f"Country:  
{six_item_dict['country']}") print(f"Email:  
{six_item_dict['email']}")
```

Q2. Write a Python program to access values using dictionary keys.

```
def access_dictionary_values(my_dict,  
keys_to_access): """ Accesses and prints  
values from a dictionary using a list of  
keys. Args: my_dict: The dictionary to  
access. keys_to_access: A list of keys to  
retrieve values for. """ for key in  
keys_to_access: if key in my_dict: value =  
my_dict[key] print(f"Value for key '{key}':  
{value}") else: print(f"Key '{key}' not found  
in the dictionary.") # Example usage:  
my_dictionary = { "name": "Bob", "age":  
25, "city": "London", "job": "Software  
Developer" } keys_to_retrieve = ["name",  
"age", "country", "job"] # Include a non-  
existent key  
access_dictionary_values(my_dictionary,  
keys_to_retrieve) #Another example with  
different datatypes. my_dictionary2 = { 1:  
"one", 3.14: "pi", True: "boolean", "list":  
[1,2,3] } keys_to_retrieve2 = [1, 3.14,  
False, "list"]
```

```
access_dictionary_values(my_dictionary2  
, keys_to_retrieve2)
```

Practical Examples:

Q1. Write a Python program to create a dictionary of 6 key-value pairs.

```
def create_six_key_value_dictionary():  
    """Creates a dictionary with 6 key-value  
    pairs.""" my_dictionary = { "apple": 1,  
    "banana": 2, "cherry": 3, "date": 4,  
    "elderberry": 5, "fig": 6 } return  
my_dictionary # Example usage: my_dict =  
create_six_key_value_dictionary()  
print(my_dict) # Accessing values from  
the dictionary: print(f"Value for 'apple':  
{my_dict['apple']}") print(f"Value for  
'cherry': {my_dict['cherry']}") #Another  
example with different datatypes. def  
create_six_mixed_dictionary(): mixed_dict  
= { "name": "John", "age": 30, 3.14: "pi",  
True: "boolean", "list": [1,2,3], "tuple":  
(4,5,6) } return mixed_dict  
mixed_dictionary =
```

```
create_six_mixed_dictionary()
print(mixed_dictionary) print(f"Name:
{mixed_dictionary['name']}") print(f"Pi:
{mixed_dictionary[3.14]}") print(f"List:
{mixed_dictionary['list']}")
```

Q2. Write a Python program to access values using keys from a dictionary.

```
def access_values_by_keys(my_dict,
keys): """ Accesses and prints values
from a dictionary using a list of keys.
Args: my_dict: The dictionary to access.
keys: A list of keys to retrieve values for.
""" for key in keys: if key in my_dict: value
= my_dict[key] print(f"Value for key
'{key}': {value}") else: print(f"Key '{key}'
not found in the dictionary.") # Example
usage: my_dictionary = { "name": "Alice",
"age": 30, "city": "New York", "job":
"Software Engineer" } keys_to_access =
["name", "age", "country", "job"] #country
is not in the dictionary.
access_values_by_keys(my_dictionary,
keys_to_access) #Another Example with
```

```
different data types. mixed_dictionary = {
1 : "one", 3.14 : "pi", True : "true", "list" :
[1,2,3] } mixed_keys = [1, 3.14, False,
"list"] #False is not in the dictionary.
access_values_by_keys(mixed_dictionary,
mixed_keys) #Example using the get()
method. def
access_values_with_get(my_dict, keys):
for key in keys: print(f"Value for key
'{key}': {my_dict.get(key, 'Key not
found')}")
access_values_with_get(my_dictionary,
keys_to_access)
```

7. Working with Dictionaries

Theory:

Q1. Iterating over a dictionary using loops

```
def iterate_dictionary(my_dict): """
Iterates over a dictionary and prints its
keys and values. Args: my_dict: The
dictionary to iterate over. """
print("Iterating over keys and values:") for
key, value in my_dict.items(): print(f"Key:
```

```

{key}, Value: {value}") print("\nIterating
over keys only:") for key in my_dict.keys():
print(f"Key: {key}") print("\nIterating over
values only:") for value in
my_dict.values(): print(f"Value: {value}") #
Example usage: my_dictionary = { "name":
"Charlie", "age": 35, "city": "Berlin",
"job": "Data Scientist" }
iterate_dictionary(my_dictionary)
#Another Example with different
datatypes. mixed_dictionary = { 1: "one",
3.14: "pi", True: "boolean", "list": [1,2,3] }
iterate_dictionary(mixed_dictionary)

```

Q2. Merging two lists into a dictionary using loops or zip().

```

def merge_lists_to_dict_loop(keys,
values):

```

```

    """

```

Merges two lists into a dictionary using a loop.

Args:

keys: The list of keys.

values: The list of values.

Returns:

A dictionary created from the two lists.

"""

```
merged_dict = {}
```

```
for i in range(min(len(keys),  
len(values))): #Iterate up to the shortest  
length.
```

```
    merged_dict[keys[i]] = values[i]
```

```
return merged_dict
```

Example usage:

```
keys_list = ["name", "age", "city"]
```

```
values_list = ["Alice", 30, "New York"]
```

```
result_dict =  
merge_lists_to_dict_loop(keys_list,  
values_list)  
  
print(f"Merged dictionary (loop):  
{result_dict}")
```

```
keys_list2 = ["a", "b", "c","d"]  
values_list2 = [1,2,3]
```

```
result_dict2 =  
merge_lists_to_dict_loop(keys_list2,  
values_list2)  
  
print(f"Merged dictionary (loop, different  
lengths): {result_dict2}")
```

2. Using zip():

Python

```
def merge_lists_to_dict_zip(keys, values):  
    """
```

Merges two lists into a dictionary using zip().

Args:

keys: The list of keys.

values: The list of values.

Returns:

A dictionary created from the two lists.

"""

merged_dict = dict(zip(keys, values))

return merged_dict

Example usage:

keys_list = ["name", "age", "city"]

values_list = ["Alice", 30, "New York"]

```
result_dict =  
merge_lists_to_dict_zip(keys_list,  
values_list)  
print(f"Merged dictionary (zip):  
{result_dict}")
```

```
keys_list2 = ["a", "b", "c","d"]  
values_list2 = [1,2,3]
```

```
result_dict2 =  
merge_lists_to_dict_zip(keys_list2,  
values_list2)  
print(f"Merged dictionary (zip, different  
lengths): {result_dict2}")
```

Q3. Counting occurrences of characters in a string using dictionaries.

```
def  
count_character_occurrences(input_string):  
    """ Counts the occurrences of each  
    character in a string using a dictionary.  
    Args: input_string: The string to analyze.
```

Returns: A dictionary where keys are characters and values are their counts.

```
""" char_counts = {} # Initialize an empty
dictionary for char in input_string: if char
in char_counts: char_counts[char] += 1 #
Increment count if character exists else:
char_counts[char] = 1 # Add character
with count 1 if it's new return char_counts
# Example usage: my_string = "hello
world" counts =
count_character_occurrences(my_string)
print(counts) my_string2 =
"programming" counts2 =
count_character_occurrences(my_string2
) print(counts2) my_string3 = "" #empty
string. counts3 =
count_character_occurrences(my_string3
) print(counts3) my_string4 =
"aaaaabbbbccccdddd" #test with many
repeated characters. counts4 =
count_character_occurrences(my_string4
) print(counts4)
```

Lab:

Q1. Write a Python program to update a value in a dictionary

```
def update_dictionary_value(my_dict, key, new_value): """ Updates the value associated with a given key in a dictionary. Args: my_dict: The dictionary to update. key: The key whose value needs to be updated. new_value: The new value to assign to the key. Returns: The updated dictionary. """ if key in my_dict: my_dict[key] = new_value print(f"Value for key '{key}' updated to: {new_value}") else: print(f"Key '{key}' not found in the dictionary.") return my_dict # Example usage: my_dictionary = { "name": "Alice", "age": 30, "city": "New York" } print(f"Original dictionary: {my_dictionary}") # Update an existing key updated_dict1 = update_dictionary_value(my_dictionary, "age", 31) print(f"Updated dictionary: {updated_dict1}") # Update a non-existent key updated_dict2 =
```

```
update_dictionary_value(my_dictionary,
"country", "USA") print(f"Updated
dictionary: {updated_dict2}") #Another
example with different datatypes.
mixed_dict = { 1 : "one", 3.14 : "pi", True :
"true" } print(f"Original mixed dictionary:
{mixed_dict}") updated_mixed_dict =
update_dictionary_value(mixed_dict, 3.14,
"3.14159") print(f"Updated mixed
dictionary: {updated_mixed_dict}")
updated_mixed_dict2 =
update_dictionary_value(mixed_dict,
"test", "new value") print(f"Updated
mixed dictionary:
{updated_mixed_dict2}")
```

Q2. Write a Python program to merge two lists into one dictionary using a loop.

```
def merge_lists_to_dictionary(keys_list,
values_list): """ Merges two lists into a
dictionary using a loop. Args: keys_list:
The list of keys. values_list: The list of
values. Returns: A dictionary created
from the two lists. """ merged_dict = {}
```

```
min_length = min(len(keys_list),
len(values_list)) #Finds the minimum
length of the two lists. for i in
range(min_length):
merged_dict[keys_list[i]] = values_list[i]
return merged_dict # Example usage:
keys1 = ["name", "age", "city"] values1 =
["Alice", 30, "New York"] result1 =
merge_lists_to_dictionary(keys1, values1)
print(f"Merged dictionary 1: {result1}")
keys2 = ["a", "b", "c", "d"] values2 = [1, 2,
3] # Shorter values list result2 =
merge_lists_to_dictionary(keys2, values2)
print(f"Merged dictionary 2 (different
lengths): {result2}") keys3 = [] values3 = []
result3 = merge_lists_to_dictionary(keys3,
values3) print(f"Merged dictionary 3
(empty lists): {result3}") keys4 = ["one",
"two"] values4 = [True, 3.14] result4 =
merge_lists_to_dictionary(keys4, values4)
print(f"Merged dictionary 4 (different
types): {result4}")
```

Practical Examples:

Q1. Write a Python program to update a value at a particular key in a dictionary

```
def update_dictionary_value(my_dict, key, new_value): """ Updates the value at a particular key in a dictionary. Args: my_dict: The dictionary to update. key: The key whose value needs to be updated. new_value: The new value to assign to the key. Returns: The updated dictionary. """ if key in my_dict: my_dict[key] = new_value print(f"Value at key '{key}' updated to: {new_value}") else: print(f"Key '{key}' not found in the dictionary.") return my_dict # Example usage: my_dictionary = { "name": "Alice", "age": 30, "city": "New York" } print(f"Original dictionary: {my_dictionary}") # Update an existing key updated_dict1 = update_dictionary_value(my_dictionary, "age", 31) print(f"Updated dictionary: {updated_dict1}") # Update a non-existent key updated_dict2 =
```

```
update_dictionary_value(my_dictionary,
"country", "USA") print(f"Updated
dictionary: {updated_dict2}") #Another
example with mixed datatypes. mixed_dict
= { 1 : "one", 3.14 : "pi", True : "true", "list"
: [1,2,3] } print(f"Original mixed dictionary:
{mixed_dict}") updated_mixed_dict =
update_dictionary_value(mixed_dict, 3.14,
"3.14159") print(f"Updated mixed
dictionary: {updated_mixed_dict}")
updated_mixed_dict2 =
update_dictionary_value(mixed_dict,
"test", "new value") print(f"Updated
mixed dictionary:
{updated_mixed_dict2}")
updated_mixed_dict3 =
update_dictionary_value(mixed_dict,
"list", [4,5,6]) print(f"Updated mixed
dictionary: {updated_mixed_dict3}")
```

Q2. Write a Python program to separate keys and values from a dictionary using keys() and values() methods.

```

def separate_keys_values(my_dict): """
Separates keys and values from a
dictionary using keys() and values()
methods. Args: my_dict: The dictionary to
separate. """ keys = list(my_dict.keys()) #
Get keys as a list values =
list(my_dict.values()) # Get values as a list
print("Keys:", keys) print("Values:",
values) # Example usage: my_dictionary =
{ "name": "Alice", "age": 30, "city": "New
York", "job": "Engineer" }
separate_keys_values(my_dictionary)
#Another example with different
datatypes. mixed_dict = { 1: "one", 3.14:
"pi", True: "boolean", "list": [1, 2, 3] }
separate_keys_values(mixed_dict)
empty_dict = {}
separate_keys_values(empty_dict)

```

Q3. Write a Python program to convert two lists into one dictionary using a for loop.

```

def lists_to_dictionary_loop(keys_list,
values_list): """ Converts two lists into a

```

dictionary using a for loop. Args:
keys_list: The list of keys. values_list: The list of values. Returns: A dictionary created from the two lists. """
result_dict = {}
min_length = min(len(keys_list), len(values_list))
for i in range(min_length):
 result_dict[keys_list[i]] = values_list[i]
return result_dict
Example usage:
keys1 = ["a", "b", "c"]
values1 = [1, 2, 3]
dictionary1 = lists_to_dictionary_loop(keys1, values1)
print(f"Dictionary 1: {dictionary1}")
keys2 = ["name", "age", "city", "country"]
values2 = ["Alice", 30, "New York"]
Shorter values list
dictionary2 = lists_to_dictionary_loop(keys2, values2)
print(f"Dictionary 2 (different lengths): {dictionary2}")
keys3 = []
values3 = []
dictionary3 = lists_to_dictionary_loop(keys3, values3)
print(f"Dictionary 3 (empty lists): {dictionary3}")
keys4 = [1, 2, 3]
values4 = ["one", "two", "three"]
dictionary4 =

```
lists_to_dictionary_loop(keys4,values4)
print(f"Dictionary 4 (different types):
{dictionary4}")
```

Q4. Write a Python program to count how many times each character appears in a string.

```
def
count_character_frequency(input_string):
""" Counts the frequency of each
character in a string. Args: input_string:
The string to analyze. Returns: A
dictionary where keys are characters and
values are their frequencies. """
char_frequency = {} for char in
input_string: if char in char_frequency:
char_frequency[char] += 1 else:
char_frequency[char] = 1 return
char_frequency # Example usage: string1
= "hello world" frequency1 =
count_character_frequency(string1)
print(f"Character frequency in '{string1}':
{frequency1}") string2 = "programming is
fun" frequency2 =
```

```
count_character_frequency(string2)
print(f"Character frequency in '{string2}':
{frequency2}") string3 = "" #empty string
frequency3 =
count_character_frequency(string3)
print(f"Character frequency in '{string3}':
{frequency3}") string4 =
"aaaaabbbbccccdddd" #test with many
repeated characters. frequency4 =
count_character_frequency(string4)
print(f"Character frequency in '{string4}':
{frequency4}")
```

8. Functions

Theory:

Q1. Defining functions in Python.

Defining functions in Python is a fundamental concept for writing organized, reusable, and modular code. Here's a breakdown of how it works:

Basic Function Definition:

- **def keyword:**

- Functions in Python are defined using the `def` keyword.
- **Function name:**
 - Followed by the function name (which should adhere to Python's naming conventions).
- **Parentheses ():**
 - The function name is followed by parentheses, which may contain parameters (inputs) to the function.
- **Colon ::**
 - The parentheses are followed by a colon, which marks the beginning of the function's code block.
- **Indentation:**
 - The function's code block is indented. Python uses indentation to define code blocks.
- **return statement (optional):**
 - The return statement is used to return a value from the function. If

no return statement is present, the function returns None.

Syntax:

Python

```
def function_name(parameter1,  
parameter2, ...):
```

```
    """Docstring explaining the function's  
    purpose."""
```

```
    # Function code block
```

```
    # ...
```

```
    return value # Optional return  
statement
```

Example:

Python

```
def add_numbers(x, y):
```

```
    """This function adds two numbers and  
    returns the result."""
```

```
    sum = x + y
```

```
    return sum
```

Calling the function:

```
result = add_numbers(5, 3)
```

```
print(result) # Output: 8
```

Key Concepts:

- . Parameters:**

- Parameters are variables that receive values when the function is called. They are defined within the parentheses in the function definition.

- . Arguments:**

- Arguments are the actual values that are passed to the function when it is called.

- . Docstrings:**

- Docstrings (documentation strings) are multiline strings that provide documentation for the function. They are placed immediately after the function definition.

- . Scope:**

- Variables defined within a function have local scope, meaning they are only accessible within the function.
- Variables defined outside of a function have global scope, and can be accessed within the function, unless a local variable with the same name is defined.
- **Return Values:**
 - Functions can return values using the return statement.
 - If a function does not have a return statement, it implicitly returns None.

Types of Arguments:

- **Positional arguments:**
 - Arguments are passed to the function in the order they are defined.
- **Keyword arguments:**

- Arguments are passed to the function using their parameter names.
- **Default arguments:**
 - Parameters can have default values, which are used if no argument is passed for that parameter.
- **Variable-length arguments:**
 - Functions can accept a variable number of arguments using `*args` (for positional arguments) or `**kwargs` (for keyword arguments).

Q2. Different types of functions:
with/without parameters, with/without return values.

In Python, functions can be categorized based on whether they accept parameters and whether they return values. Here's a breakdown of the different types:

1. Functions Without Parameters and Without Return Values:

- These functions perform a specific task without requiring any input and without returning any output.
- They often perform actions like printing messages or modifying global variables.

Python

```
def greet():  
    """Prints a greeting message."""  
    print("Hello, world!")
```

```
greet() # Calling the function
```

2. Functions With Parameters and Without Return Values:

- These functions accept input values (parameters) and perform operations using those values.

- . They don't explicitly return any value, so they implicitly return None.

Python

```
def print_sum(a, b):  
    """Prints the sum of two numbers."""  
    print(f"The sum is: {a + b}")
```

`print_sum(5, 3)` # Calling the function with arguments

3. Functions Without Parameters and With Return Values:

- . These functions perform a task and return a value, but they don't require any input parameters.
- . They might retrieve data from an external source or perform calculations based on internal state.

Python

```
def get_current_time():
```

```
"""Returns the current time  
(simplified example)."""
```

```
    return "12:00 PM" # In a real  
scenario, use datetime module
```

```
current_time = get_current_time()  
print(f"Current time: {current_time}")
```

4. Functions With Parameters and With Return Values:

- These functions accept input values, perform operations, and return a result.
- This is the most common and versatile type of function.

Python

```
def add(x, y):  
    """Returns the sum of two  
numbers."""  
    return x + y
```

```
result = add(10, 20)
```

```
print(f"Result: {result}")
```

Q3. Anonymous functions (lambda functions).

Anonymous functions, also known as lambda functions, are small, single-expression functions in Python that don't require a formal name. They are defined using the lambda keyword.

Syntax:

Python

lambda arguments: expression

- . lambda:** The keyword used to define an anonymous function.
- . arguments:** The input parameters of the function, separated by commas (similar to regular function parameters).
- . expression:** A single expression that is evaluated and returned by the function. Lambda functions can only

contain expressions, not statements
(like if, for, while, etc.).

Key Characteristics:

- **Anonymous:** They don't have a name like regular functions defined with def.
- **Single Expression:** They can only contain a single expression.
- **Implicit Return:** The result of the expression is automatically returned.
- **Concise:** They are often used for short, simple operations.

Examples:

1. Adding two numbers:

Python

```
add = lambda x, y: x + y  
result = add(5, 3)  
print(result) # Output: 8
```

2. Squaring a number:

Python

```
square = lambda x: x * x
```

```
result = square(4)
```

```
print(result) # Output: 16
```

3. Checking if a number is even:

Python

```
is_even = lambda x: x % 2 == 0
```

```
result = is_even(7)
```

```
print(result) # Output: False
```

4. Using lambda with map():

Python

```
numbers = [1, 2, 3, 4, 5]
```

```
squared_numbers = list(map(lambda x:  
x * x, numbers))
```

```
print(squared_numbers) # Output: [1, 4,  
9, 16, 25]
```

5. Using lambda with filter():

Python

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = list(filter(lambda x: x %  
2 == 0, numbers))
```

```
print(even_numbers) # Output: [2, 4, 6]
```

6. Using lambda with sorted():

Python

```
points = [(1, 5), (3, 2), (2, 8)]
```

```
sorted_points = sorted(points,  
key=lambda point: point[1])
```

```
print(sorted_points) #output: [(3, 2), (1,  
5), (2, 8)]
```

When to Use Lambda Functions:

- When you need a small, simple function for a short period of time.
- When you want to pass a function as an argument to another function (e.g., `map()`, `filter()`, `sorted()`).
- When you want to write concise, one-line functions.

When Not to Use Lambda Functions:

- When the function logic is complex or involves multiple statements.

- When you need to reuse the function in multiple places (in this case, define a regular function with def).
- When the function is long or hard to read. Readability is important, and complex logic should be handled with a standard function.

Lab:

Q1. Write a Python program to create a function that takes a string as input and prints it.

```
def print_string(input_string):
```

```
    """
```

```
    Takes a string as input and prints it.
```

```
    Args:
```

```
        input_string: The string to print.
```

```
    """
```

```
    print(input_string)
```

Example usage:

```
my_string = "Hello, world!"
```

```
print_string(my_string)
```

```
another_string = "This is a test."
```

```
print_string(another_string)
```

```
empty_string = ""
```

```
print_string(empty_string)
```

```
number_string = "12345"
```

```
print_string(number_string)
```

Explanation:

- 1. def print_string(input_string)::**
 - This line defines a function named **print_string**.
 - It takes one parameter, **input_string**, which will hold the string passed to the function.

2. `"""..."""`:

- This is a docstring (documentation string). It's good practice to include docstrings to explain what your functions do.

3. `print(input_string)`:

- This line uses the built-in `print()` function to display the value of `input_string` to the console.

4. Example Usage:

- Several examples are provided to show how the function works with different strings.
- `my_string`, `another_string`, `empty_string`, and `number_string` are created to test various cases.
- The `print_string()` function is called with each of these strings, and the output is displayed.

Q2. Write a Python program to create a calculator using functions.

```
def add(x, y): """Adds two numbers."""
return x + y
def subtract(x, y):
"""Subtracts two numbers."""
return x - y
def multiply(x, y): """Multiplies two
numbers."""
return x * y
def divide(x, y):
"""Divides two numbers."""
if y == 0:
return "Cannot divide by zero"
return x / y
def calculator(): """A simple calculator
program."""
print("Select operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Divide")
while True:
try:
choice = input("Enter choice
(1/2/3/4): ")
num1 = float(input("Enter first
number: "))
num2 = float(input("Enter
second number: "))
if choice == '1':
print(num1, "+", num2, "=", add(num1,
num2))
elif choice == '2':
print(num1, "-", num2, "=", subtract(num1, num2))
elif choice == '3':
print(num1, "*", num2, "=", multiply(num1, num2))
elif choice == '4':
print(num1, "/", num2, "=", divide(num1, num2))
else:
print("Invalid input")
break
#Break out of the while loop after one
```

```
calculation. except ValueError:  
print("Invalid input. Please enter  
numbers.") except Exception as e:  
print(f"An error occurred: {e}")  
calculator()
```

Practical Examples:

Q1. Write a Python program to print a string using a function.

```
def print_given_string(input_string):  
    """Prints the given string to the console.  
    Args: input_string: The string to be  
    printed. """  
    print(input_string) # Example  
Usage: my_string = "Hello, world!"  
print_given_string(my_string)  
another_string = "Python is fun!"  
print_given_string(another_string)  
empty_string = ""  
print_given_string(empty_string)  
number_string = "12345"  
print_given_string(number_string)  
mixed_string = "This string has numbers
```

123 and symbols !@#."

```
print_given_string(mixed_string)
```

Q2. Write a Python program to create a parameterized function that takes two arguments and prints their sum

```
def print_sum(num1, num2): """
```

```
Calculates and prints the sum of two  
numbers. Args: num1: The first number.
```

```
num2: The second number. """ sum_result
```

```
= num1 + num2 print(f"The sum of {num1}  
and {num2} is: {sum_result}") # Example
```

```
usage: print_sum(5, 10) print_sum(15.5,  
20.2) print_sum(-3, 8) print_sum(0,0)
```

Q3. Write a Python program to create a lambda function with one expression.

```
# Lambda function to square a number
```

```
square = lambda x: x * x # Lambda
```

```
function to check if a number is even
```

```
is_even = lambda x: x % 2 == 0 # Lambda
```

```
function to add 10 to a number add_10 =
```

```
lambda x: x + 10 # Lambda function to
```

```
convert a string to uppercase to_upper =
```

```
lambda s: s.upper() # Lambda function to
return the first character of a string.
first_char = lambda s: s[0] if s else None
#Handles empty strings. # Example usage:
print(square(5)) # Output: 25
print(is_even(7)) # Output: False
print(add_10(3)) # Output: 13
print(to_upper("hello")) # Output: HELLO
print(first_char("python")) #output: p
print(first_char("")) #output: None
```

Q4. Write a Python program to create a lambda function with two expressions

Lambda functions in Python are designed to have a single expression. If you try to put two separate expressions directly into a lambda, you'll get a syntax error.

However, you can achieve a similar effect by using a single expression that combines multiple operations, like a conditional expression or a tuple.

Here are a few ways to work around the single-expression limitation, though they

don't truly execute two independent expressions sequentially:

1. Using a Conditional Expression (for branching logic):

Python

Lambda function to return one of two values based on a condition

result = lambda x, y: x * y if x > y else x + y

print(result(5, 3)) # Output: 15 (5 * 3)

print(result(2, 4)) # Output: 6 (2 + 4)

In this example, the lambda function effectively performs two different calculations based on the condition $x > y$, but it's still a single conditional expression.

2. Using a Tuple (for returning multiple values):

Python

Lambda function to return a tuple containing two calculated values

```
result = lambda x, y: (x * y, x + y)
```

```
print(result(5, 3)) # Output: (15, 8)
```

Here, the lambda function calculates two values and returns them as a tuple. While it appears to be doing two things, it's returning one tuple which contains two values.

3. Using a function within a function (for more complex logic):

If you need to perform more complex operations or multiple statements, it's best to use a regular def function. If you must use a lambda, you could technically call another function from within the lambda.

Python

```
def multiply_and_add(x, y):
```

```
    return x * y, x + y
```

```
result = lambda x, y: multiply_and_add(x,  
y)
```

```
print(result(5, 3)) #output: (15, 8)
```

9. Modules

Theory:

Q1. Introduction to Python modules and importing modules.

What are Modules?

- . A module is simply a Python file (.py extension) containing code that you can use in other Python programs.**
- . Modules can contain functions, classes, variables, and executable code.**
- . They provide a way to break down large programs into smaller, more manageable pieces.**

- Modules promote code reusability by allowing you to use the same code in multiple programs.
- Python has a vast standard library of modules that provide a wide range of functionality.

Creating Modules:

To create a module, simply create a Python file (e.g., `my_module.py`) and write your code in it.

Python

```
# my_module.py
```

```
def greet(name):
```

```
    """Greets a person by name."""
```

```
    print(f"Hello, {name}!")
```

```
pi = 3.14159
```

```
class MyClass:
```

```
def __init__(self, value):  
    self.value = value
```

```
def display(self):  
    print(f"Value: {self.value}")
```

Importing Modules:

To use a module in another Python program, you need to import it. There are several ways to import modules:

1. **import module_name:**
 - This imports the entire module.
 - You access the module's contents using the dot notation (module_name.function_name, module_name.variable_name, etc.).

Python

main.py

import my_module

```
my_module.greet("Alice")  
print(my_module.pi)
```

```
obj = my_module.MyClass(10)  
obj.display()
```

2. **import module_name as alias:**
 - This imports the module and assigns it an alias (a shorter name).
 - This is useful for long module names or to avoid naming conflicts.

Python

main.py

```
import my_module as mm
```

```
mm.greet("Bob")  
print(mm.pi)
```

3. **from module_name import name1, name2, ...:**

- This imports specific names (functions, classes, variables) from the module.
- You can then use those names directly without the module name prefix.

Python

main.py

from my_module import greet, pi

greet("Charlie")

print(pi)

4. from module_name import *:

- This imports all names from the module.
- While convenient, it's generally discouraged because it can lead to naming conflicts and make your code less readable.

Python

```
# main.py
from my_module import *
```

```
greet("David")
print(pi)
obj = MyClass(20)
obj.display()
```

Module Search Path:

When you import a module, Python searches for it in a specific order:

1. The current directory.
2. Directories listed in the PYTHONPATH environment variable.
3. Installation-dependent default directories.

You can inspect the module search path using `sys.path`:

Python

```
import sys
```

```
print(sys.path)
```

Key Advantages of Modules:

- **Code Organization:** Modules help you organize your code into logical units.
- **Code Reusability:** You can reuse code from modules in multiple programs.
- **Namespace Management:** Modules create separate namespaces, preventing naming conflicts.
- **Modularity:** Modules promote modular programming, making your code easier to maintain and debug.

Q2. Standard library modules: math, random.

Python's standard library is a treasure trove of pre-built modules that provide a wide range of functionalities. Among the most frequently used are the math and random modules. Here's a closer look at them:

1. math Module:

- The math module provides access to a wide range of mathematical functions and constants.
- It's incredibly useful for performing complex mathematical calculations.

Key Features:

- **Constants:**
 - `math.pi`: The mathematical constant π (pi).
 - `math.e`: The mathematical constant e .
- **Mathematical Functions:**
 - `math.sqrt(x)`: Returns the square root of x .
 - `math.pow(x, y)`: Returns x raised to the power of y .
 - `math.sin(x)`, `math.cos(x)`, `math.tan(x)`: Trigonometric functions.
 - `math.log(x[, base])`: Returns the logarithm of x to the given base.

- `math.ceil(x)`: Returns the smallest integer greater than or equal to `x`.
- `math.floor(x)`: Returns the largest integer less than or equal to `x`.¹

[1. www.brainscape.com](http://www.brainscape.com)

www.brainscape.com

Example:

Python

```
import math
```

```
print(math.pi)
```

```
print(math.sqrt(16))
```

```
print(math.pow(2, 3))
```

```
print(math.sin(math.pi / 2))
```

2. random Module:

- The random module provides functions for generating pseudo-random numbers.
- It's essential for tasks like simulations, games, and security.

Key Features:

- **Generating Random Numbers:**
 - `random.random()`: Returns a random float between 0 and 1.
 - `random.randint(a, b)`: Returns a random integer between a and b (inclusive).
 - `random.randrange(start, stop[, step])`: Returns a randomly selected element from the range created by the start, stop and step arguments.
- **Working with Sequences:**
 - `random.choice(seq)`: Returns a random element from a sequence.

- `random.shuffle(seq)`: Shuffles the elements of a sequence in place.
- **Seeding:**
 - `random.seed(x)`: Initializes the random number generator. This is useful for reproducing random sequences.

Example:

Python

```
import random
```

```
print(random.random())
```

```
print(random.randint(1, 10))
```

```
my_list = [1, 2, 3, 4, 5]
```

```
random.shuffle(my_list)
```

```
print(my_list)
```

```
print(random.choice(my_list))
```

Q3. Creating custom modules.

Creating custom modules in Python is straightforward and allows you to

organize your code into reusable components. Here's a step-by-step guide:

1. Create a Python File:

- A module is simply a Python file (.py extension).
- Create a new Python file in your project directory. For example, let's create a module named `my_module.py`.

2. Add Code to the Module:

- Inside `my_module.py`, you can define functions, classes, variables, or any other Python code you want to include in your module.

Python

```
# my_module.py
```

```
def greet(name):
```

```
    """Greets a person by name."""
```

```
    return f"Hello, {name}!"
```

```
def square(x):  
    """Returns the square of a number."""  
    return x * x  
  
my_variable = "This is a module variable."  
  
class MyClass:  
    def __init__(self, value):  
        self.value = value  
  
    def display(self):  
        return f"Value: {self.value}"
```

3. Use the Module in Another Python File:

- Create another Python file (e.g., main.py) in the same directory as my_module.py.
- Import your custom module using the import statement.

Python

main.py

import my_module

Use functions from the module

greeting = my_module.greet("Alice")

print(greeting)

squared_value = my_module.square(5)

print(squared_value)

Access variables from the module

print(my_module.my_variable)

Use classes from the module

obj = my_module.MyClass(10)

print(obj.display())

#Or import specific functions.

from my_module import greet

print(greet("Bob"))

4. Module Search Path:

- . When you import a module, Python searches for it in a specific order:**
 - 1. The current directory.**
 - 2. Directories listed in the PYTHONPATH environment variable.**
 - 3. Installation-dependent default directories.**
- . If your module is in a different directory, you'll need to add that directory to the module search path. You can do this programmatically:**

Python

import sys

import os

```
module_path =  
os.path.abspath("path/to/your/module/directory") #replace with the actual path.  
sys.path.append(module_path)
```

```
import my_module #Now you can import it.
```

5. Packages (Organizing Modules):

- For larger projects, you can organize modules into packages.
- A package is a directory containing Python modules and a special file named `__init__.py`.
- The `__init__.py` file can be empty, or it can contain initialization code for the package.
- Example package structure:

```
my_package/  
    __init__.py
```

module1.py

module2.py

- To import modules from a package:

Python

```
import my_package.module1
```

```
from my_package import module2
```

Best Practices:

- **Descriptive Names:** Use meaningful names for your modules, functions, classes, and variables.
- **Docstrings:** Add docstrings to your modules, functions, and classes to provide documentation.
- **Organization:** Organize your code into logical modules and packages.
- **Avoid Circular Imports:** Be careful to avoid circular imports, which can lead to errors.
- **Use `if __name__ == "__main__":`:** If your module contains executable code, use

if `__name__ == "__main__"`: to prevent it from running when the module is imported.

Python

`# my_module.py`

`def my_function():`

`# ...`

`if __name__ == "__main__":`

`# Code to run when the module is
 executed as a script`

`my_function()`

Lab:

Q1. Write a Python program to import the math module and use functions like `sqrt()`, `ceil()`, `floor()`.

`import math` `def`

`use_math_functions(number): """`

Demonstrates the use of math module

functions: sqrt(), ceil(), and floor(). Args:
number: The number to use with the math
functions. """ print(f"Original number:
{number}") # Square root sqrt_result =
math.sqrt(number) print(f"Square root:
{sqrt_result}") # Ceiling (smallest integer
greater than or equal to the number)
ceil_result = math.ceil(number)
print(f"Ceiling: {ceil_result}") # Floor
(largest integer less than or equal to the
number) floor_result = math.floor(number)
print(f"Floor: {floor_result}") # Example
usage: use_math_functions(16)
use_math_functions(7.3)
use_math_functions(9.999)
use_math_functions(25)
use_math_functions(2)

**Q2. Write a Python program to generate
random numbers using the random
module.**

```
import random  
def generate_random_numbers():  
    """Generates and prints various random
```

```
numbers using the random module. """ #
Random float between 0 and 1
random_float = random.random()
print(f"Random float (0 to 1):
{random_float}") # Random integer within
a range random_int = random.randint(1,
10) # Inclusive range print(f"Random
integer (1 to 10): {random_int}") # Random
number from a range with a step
random_range = random.randrange(0,
100, 5) #0, 5, 10, ... 95 print(f"Random
number from range(0, 100, 5):
{random_range}") # Random choice from
a list my_list = [10, 20, 30, 40, 50]
random_choice = random.choice(my_list)
print(f"Random choice from list:
{random_choice}") # Shuffle a list in place
random.shuffle(my_list) print(f"Shuffled
list: {my_list}") #Generate a list of random
numbers random_list =
[random.randint(1,100) for _ in range(5)]
print(f"List of random numbers:
{random_list}") #Generate a random float
```

```
between two values. random_float_range
= random.uniform(2.5, 7.5)
print(f"Random float between 2.5 and 7.5:
{random_float_range}") # Example usage:
generate_random_numbers() #Seeding
the random number generator.
random.seed(42) #Seed with a constant
value. print(random.randint(1,10)) #This
will always produce the same random
number. random.seed(42) #Seed again
with the same value.
print(random.randint(1,10)) #This will
produce the same number as the previous
call.
```

Practical Examples:

Q1. Write a Python program to demonstrate the use of functions from the math module.

```
import math def
demonstrate_math_functions():
"""Demonstrates various functions from
the math module.""" # Constants
```

```
print(f"Pi: {math.pi}") print(f"Euler's  
number (e): {math.e}") # Basic arithmetic  
print(f"Square root of 16:  
{math.sqrt(16)}") print(f"2 raised to the  
power of 3: {math.pow(2, 3)}") #  
Trigonometric functions (radians)  
angle_radians = math.pi / 2 # 90 degrees  
print(f"Sine of pi/2:  
{math.sin(angle_radians)}") print(f"Cosine  
of pi/2: {math.cos(angle_radians)}")  
print(f"Tangent of pi/2:  
{math.tan(angle_radians)}") # Logarithmic  
functions print(f"Natural logarithm of e:  
{math.log(math.e)}") print(f"Base-10  
logarithm of 100: {math.log10(100)}") #  
Rounding functions print(f"Ceiling of 3.14:  
{math.ceil(3.14)}") print(f"Floor of 3.99:  
{math.floor(3.99)}") #Absolute value  
print(f"Absolute value of -10: {math.fabs(-  
10)}") #Factorial print(f"Factorial of 5:  
{math.factorial(5)}") #Degrees and  
Radians conversion. degrees = 180  
radians = math.radians(degrees)
```

```
print(f"{degrees} degrees is equal to  
{radians} radians") radians = math.pi  
degrees = math.degrees(radians)  
print(f"{radians} radians is equal to  
{degrees} degrees") # Call the  
demonstration function  
demonstrate_math_functions()
```

Q2. Write a Python program to generate random numbers between 1 and 100 using the random module

```
import random  
def generate_random_numbers(count):  
    """  
    Generates a specified number of random  
    integers between 1 and 100 (inclusive).  
    Args: count: The number of random  
    numbers to generate. """  
    if count <= 0:  
        print("Please enter a positive count.")  
        return  
    for _ in range(count):  
        random_number = random.randint(1, 100)  
        print(random_number) # Example usage:  
num_numbers = 5 # Generate 5 random  
numbers  
generate_random_numbers(num_number
```

```
s) num_numbers2 = 10 #generate 10
random numbers
generate_random_numbers(num_number
s2) generate_random_numbers(0) #test
zero case. generate_random_numbers(-3)
#test negative case.
```